AMS 526: Numerical Linear Algebra for Computational and Data Sciences

Lecture 11: Gram–Schmidt Orthogonalization; Householder Triangularization

Prof. Xiangmin Jiao

Stony Brook University

Outline

1. **Gram–Schmidt Orthogonalization (NLA §7)**
2. Modified Gram–Schmidt Orthogonalization (NLA §8)
3. Householder Triangularization (NLA §10)

Motivation

Question: Given a linear system $Ax \approx b$ where $A \in \mathbb{R}^{m\times n}$ ($m \geq n$) has full rank, how to solve the linear system?

1. One approach is to solve the normal equation $A^\top Ax = A^\top b$ directly using Cholesky factorization. It is unstable, but is very efficient if $m \gg n$ ($\sim mn^2 + \frac{1}{3}n^3$ flops).
2. Another possible solution is to use the SVD

$$A = U\Sigma V^\top, \text{ so } x = V\Sigma^{-1}U^\top b.$$

   It is stable, but is inefficient.

A more robust approach is to use QR factorization, which decomposes A into the product of two simple matrices Q and R, where the columns of Q are orthonormal and R is upper triangular.

Two Different Versions of QR

There are two versions of QR factorizations.

- The full QR factorization of $A \in \mathbb{R}^{m\times n}$ $(m \geq n)$ is

$$A = QR,$$

  where $Q \in \mathbb{R}^{m\times m}$ is orthogonal and $R \in \mathbb{R}^{m\times n}$ is upper triangular.

- The reduced (or thin) QR factorization of $A \in \mathbb{R}^{m\times n}$ $(m \geq n)$ is

$$A = \widehat{Q}\widehat{R},$$

  where $\widehat{Q} \in \mathbb{R}^{m\times n}$ contains orthonormal vectors and $\widehat{R} \in \mathbb{R}^{n\times n}$ is upper triangular.

- What space do $\{q_1, q_2, \cdots, q_j\}$, $j \leq n$ span?
  - Answer: For a full rank A, the first j column vectors of A, i.e., $\langle q_1, q_2, \ldots, q_j\rangle = \langle a_1, a_2, \ldots, a_j\rangle$.

Gram–Schmidt Orthogonalization

- A method to construct the QR factorization is to orthogonalize the column vectors of A.
- Basic idea.
  - Take the first column a_1 and normalize it to obtain the vector q_1.
  - Take the second column a_2, subtract its orthogonal projection onto q_1, and normalize to obtain q_2.
  - ...
  - Take the jth column a_j, subtract its orthogonal projection onto $q_1, \dots, q_{j-1}$, and normalize to obtain q_j as

$$v_j = a_j - \sum_{i=1}^{j-1} (q_i^\top a_j) q_i, \quad q_j = v_j / \|v_j\|.$$

- This idea is called *Gram–Schmidt orthogonalization*.

Gram–Schmidt Projections

- Orthogonal vectors produced by Gram–Schmidt can be written in terms of projectors as

$$q_j = \frac{P_j a_j}{\|P_j a_j\|},$$

where

$$P_j = I - \widehat{Q}_{j-1}\widehat{Q}_{j-1}^{\mathsf{T}} \quad \text{with} \quad \widehat{Q}_{j-1} = \begin{bmatrix} q_1 & q_2 & \cdots & q_{j-1} \end{bmatrix}.$$

- P_j projects orthogonally onto the space orthogonal to $\langle q_1, q_2, \ldots, q_{j-1} \rangle$, and the rank of P_j is $m - (j-1)$.

Existence of QR

Theorem

Every $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) has a full QR factorization, and hence also a reduced QR factorization.

Key idea of proof: If A has full rank, the Gram–Schmidt algorithm provides a constructive proof itself for having a reduced QR.

If A does not have full rank, at some step, $v_j = 0$. We can set q_j to be a vector orthogonal to q_i for $i < j$.

To construct the full QR from the reduced QR, just continue Gram–Schmidt for an additional $m - n$ steps.

Uniqueness of QR

Theorem

Every $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) of full rank has a unique reduced QR factorization $A = \widehat{Q}\widehat{R}$ with $r_{jj} > 0$.

Key idea of proof: Proof is provided by the Gram–Schmidt iteration itself. If the signs of r_{jj} are determined, then r_{ij} and q_j are determined.

Question: Why do we require $r_{jj} > 0$?

Question: Is the full QR factorization unique?

Question: What if A does not have full rank?

Outline

1. Gram–Schmidt Orthogonalization (NLA §7)
2. **Modified Gram–Schmidt Orthogonalization (NLA §8)**
3. Householder Triangularization (NLA §10)

Algorithm of Gram–Schmidt Orthogonalization

Classical Gram–Schmidt method

for $j = 1 : n$
  $v_j = a_j$
  for $i = 1 : j - 1$
    $r_{ij} = q_i^\top a_j$
    $v_j = v_j - r_{ij} q_i$
  $r_{jj} = \|v_j\|_2$
  $q_j = v_j / r_{jj}$

- GS requires $\sim 2mn^2$ flops to compute the QR factorization of an $m \times n$ matrix.
- Classical Gram–Schmidt (CGS) is **unstable**, which means that its solution is sensitive to perturbation.

Alternative View of Gram–Schmidt Projection

- Orthogonal vectors produced by Gram–Schmidt can be written in terms of projectors as

$$q_j = \frac{P_j a_j}{\|P_j a_j\|}, \quad \text{where } P_j = I - \hat{Q}_{j-1}\hat{Q}_{j-1}^{\top}, \quad \hat{Q}_{j-1} = \left[q_1 \mid q_2 \mid \cdots \mid q_{j-1}\right].$$

- We may view P_j as the product of a sequence of projections

$$P_j = P_{\perp q_{j-1}} P_{\perp q_{j-2}} \cdots P_{\perp q_1},$$

  where $P_{\perp q} = I - qq^{\top}$.
- Instead of computing $v_j = P_j a_j$, one could compute $v_j = P_{\perp q_{j-1}} P_{\perp q_{j-2}} \cdots P_{\perp q_1} a_j$ instead, resulting in the modified Gram–Schmidt algorithm.

Modified Gram–Schmidt Algorithm

Classical Gram–Schmidt method

for $j = 1 : n$
  $v_j = a_j$
  for $i = 1 : j-1$
    $r_{ij} = q_i^\top a_j$
    $v_j = v_j - r_{ij} q_i$
  $r_{jj} = \|v_j\|_2$
  $q_j = v_j / r_{jj}$

Modified Gram–Schmidt method

for $j = 1 : n$
  $v_j = a_j$
for $i = 1 : n$
  $r_{ii} = \|v_i\|_2$
  $q_i = v_i / r_{ii}$
  for $j = i+1 : n$
    $r_{ij} = q_i^\top v_j$
    $v_j = v_j - r_{ij} q_i$

- The key difference between CGS and MGS is how r_{ij} is computed.
- CGS above is column-oriented (in the sense that R is computed column by column) and MGS above is row-oriented, but this is NOT the main difference between CGS and MGS. There are also column-oriented MGS and row-oriented CGS.
- MGS is numerically “more stable” than CGS, but neither is strictly stable.
- In MGS, v_j can overwrite a_j, and q_j can overwrite v_j.

Example: CGS vs. MGS

- Consider the matrix

$$A = \begin{bmatrix} 1 & 1 & 1 \\ \varepsilon & 0 & 0 \\ 0 & \varepsilon & 0 \\ 0 & 0 & \varepsilon \end{bmatrix},$$

  where ε is small such that $1 + \varepsilon^2 = 1$ with round-off error.

- For both CGS and MGS:

$$v_1 \leftarrow (1, \varepsilon, 0, 0)^\top,\ r_{11} = \sqrt{1+\varepsilon^2} \approx 1,\ q_1 = v_1/r_{11} = (1, \varepsilon, 0, 0)^\top,$$

$$v_2 \leftarrow (1, 0, \varepsilon, 0)^\top,\ r_{12} = q_1^\top a_2 \text{ (or } = q_1^\top v_2) = 1,$$

$$v_2 \leftarrow v_2 - r_{12} q_1 = (0, -\varepsilon, \varepsilon, 0)^\top,$$

$$r_{22} = \sqrt{2}\varepsilon,\ q_2 = (0, -1, 1, 0)^\top/\sqrt{2},$$

$$v_3 \leftarrow (1, 0, 0, \varepsilon)^\top,\ r_{13} = q_1^\top a_3 \text{ (or } = q_1^\top v_3) = 1,$$

$$v_3 \leftarrow v_3 - r_{13} q_1 = (0, -\varepsilon, 0, \varepsilon)^\top.$$

Example: CGS vs. MGS Cont'd

- For CGS:

$$r_{23} = q_2^\top a_3 = 0,\ v_3 \leftarrow v_3 - r_{23} q_2 = (0, -\varepsilon, 0, \varepsilon)^\top,$$

$$r_{33} = \sqrt{2}\varepsilon,\ q_3 = v_3 / r_{33} = (0, -1, 0, 1)^\top / \sqrt{2}.$$

  - Note that $q_2^\top q_3 = (0, -1, 1, 0)(0, -1, 0, 1)^\top / 2 = 1/2$.

- For MGS:

$$r_{23} = q_2^\top v_3 = \varepsilon/\sqrt{2},\ v_3 \leftarrow v_3 - r_{23} q_2 = (0, -\varepsilon/2, -\varepsilon/2, \varepsilon)^\top,$$

$$r_{33} = \sqrt{6}\varepsilon/2,\ q_3 = v_3 / r_{33} = (0, -1, -1, 2)^\top / \sqrt{6}.$$

  - Note that $q_2^\top q_3 = (0, -1, 1, 0)(0, -1, -1, 2)^\top / \sqrt{12} = 0$.

Outline

1. Gram–Schmidt Orthogonalization (NLA §7)
2. Modified Gram–Schmidt Orthogonalization (NLA §8)
3. **Householder Triangularization (NLA §10)**

Gram–Schmidt as Triangular Orthogonalization

- Every step of Gram–Schmidt can be viewed as multiplication with a triangular matrix. For example, at the first step,

$$[v_1 \mid v_2 \mid \cdots \mid v_n] \underbrace{\begin{bmatrix} \frac{1}{r_{11}} & \frac{-r_{12}}{r_{11}} & \frac{-r_{13}}{r_{11}} & \cdots \\ & 1 & & \\ & & 1 & \\ & & & \ddots \end{bmatrix}}_{R_1} = \left[q_1 \mid v_2^{(2)} \mid \cdots \mid v_n^{(2)}\right].$$

- Gram–Schmidt therefore multiplies triangular matrices to orthogonalize column vectors, and in turn can be viewed as *triangular orthogonalization*,

$$A \underbrace{R_1 R_2 \cdots R_n}_{\hat{R}^{-1}} = \hat{Q},$$

  where R_i is a triangular matrix.
- A "dual" approach would be *orthogonal triangularization*, i.e., multiply A by orthogonal matrices to make it triangular.

Householder Triangularization

- The method was introduced by Alston Scott Householder in 1958.
- It multiplies orthogonal matrices to make it triangular, e.g.,

$$\underset{A}{\begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix}} \xrightarrow{Q_1} \underset{Q_1A}{\begin{bmatrix} \mathrm{x} & \mathrm{x} & \mathrm{x} \\ 0 & \mathrm{x} & \mathrm{x} \\ 0 & \mathrm{x} & \mathrm{x} \\ 0 & \mathrm{x} & \mathrm{x} \\ 0 & \mathrm{x} & \mathrm{x} \end{bmatrix}} \xrightarrow{Q_2} \underset{Q_2Q_1A}{\begin{bmatrix} \times & \times & \times \\ & \mathrm{x} & \mathrm{x} \\ & 0 & \mathrm{x} \\ & 0 & \mathrm{x} \\ & 0 & \mathrm{x} \end{bmatrix}} \xrightarrow{Q_3} \underset{Q_3Q_2Q_1A}{\begin{bmatrix} \times & \times & \times \\ & \times & \times \\ & & \mathrm{x} \\ & & 0 \\ & & 0 \end{bmatrix}}.$$

- After n steps, we obtain a product of orthogonal matrices

$$\underbrace{Q_n \cdots Q_2 Q_1}_{Q^\top} A = R,$$

  and in turn we get the full QR factorization $A = QR$.
- Q_k introduces zeros below the diagonal of the kth column while preserving zeros below the diagonal in preceding columns.
- The key question is how to find Q_k.

Householder Reflectors

- First, consider Q_1 such that $Q_1 a_1 = \|a_1\| e_1$, where $e_1 = (1, 0, \cdots, 0)^\top$. Why is the length $\|a_1\|$?
- Q_1 reflects a_1 across the hyperplane H orthogonal to $v = \|a_1\| e_1 - a_1$, and therefore

$$Q_1 = I - 2\frac{vv^\top}{v^\top v}.$$

- More generally,

$$Q_k = \begin{bmatrix} I & 0 \\ 0 & F \end{bmatrix},$$

  where I is $(k-1) \times (k-1)$ and F is $(m-k+1) \times (m-k+1)$ such that $Fx = \|x\|_2 e_1$, where x is $(a_{k,k}, a_{k+1,k}, \cdots, a_{m,k})^\top$.
- What is F? It has a similar form to Q_1 with $v = \|x\| e_1 - x$.

Choice of Reflectors

- We could choose F such that $Fx = -\|x\|e_1$ instead of $Fx = \|x\|e_1$.
  - More generally, $Fx = z\|x\|e_1$ with $|z| = 1$ if $z \in \mathbb{C}$.
  - This leads to an infinite number of possible QR factorizations of A.
  - If we require $z \in \mathbb{R}$, we still have two choices.
- Numerically, it is undesirable for $v^\top v$ to be close to zero for $v = z\|x\|e_1 - x$, and $\|v\|$ is larger if $z = -\operatorname{sign}(x_1)$.
- Therefore, $v = -\operatorname{sign}(x_1)\|x\|e_1 - x$. Since $I - 2\frac{vv^\top}{v^\top v}$ is independent of the sign, we can factor out the negative sign and obtain $v = \operatorname{sign}(x_1)\|x\|e_1 + x$.
- For completeness, if $x_1 = 0$, set z to 1 (instead of 0).
- We define

$$\operatorname{sign}(x) = \begin{cases} 1 & \text{if } x \geq 0, \\ -1 & \text{otherwise.} \end{cases}$$

Householder Algorithm

Householder QR Factorization

for $k = 1$ **to** n

$$x = A_{k:m,k}$$
$$v_k = \text{sign}(x_1)\|x\|e_1 + x$$
$$v_k = v_k / \|v_k\|$$
$$A_{k:m,k:n} = A_{k:m,k:n} - 2v_k(v_k^\top A_{k:m,k:n})$$

- Matrix A is overwritten by R.
- Matrix Q is not formed explicitly, but the reflection vector v_k is stored.
- Question: Can A be reused to store both R and v_k completely?
- Answer: We can use the lower triangular portion of A to store all but one entry in each v_k. An additional array of size n may be needed, or take advantage of $\|v_k\| = 1$.
- Question: What happens if v_k is 0 in line 3 of the loop?
- Answer: $r_{kk} = 0$.

Applying or Forming Q

- Compute $Q^\top b = Q_n \cdots Q_1 b$.

Implicit calculation of $Q^\top b$

for $k = 1$ **to** n

$$b_{k:m} = b_{k:m} - 2v_k(v_k^\top b_{k:m})$$

- Compute $Qx = Q_1 Q_2 \cdots Q_n x$.

Implicit calculation of Qx

for $k = n$ **downto** 1

$$x_{k:m} = x_{k:m} - 2v_k(v_k^\top x_{k:m})$$

- Question: How to form Q and $\widehat{Q}$, respectively?
- Answer: Apply $x = I_{m\times m}$ or the first n columns of I, respectively.

Operation Count

- Most work is done at the step $A_{k:m,k:n} = A_{k:m,k:n} - 2v_k(v_k^\top A_{k:m,k:n})$.
- Flops per iteration:
  - $\sim 2(m-k)(n-k)$ for dot products $v_k^\top A_{k:m,k:n}$.
  - $\sim (m-k)(n-k)$ for the outer product $(2v_k)(\cdots)$.
  - $\sim (m-k)(n-k)$ for subtraction.
  - $\sim 4(m-k)(n-k)$ total.
- Including the outer loop, the total flops count is

$$\begin{aligned}\sum_{k=1}^{n} 4(m-k)(n-k) &= 4\sum_{k=1}^{n}(mn - km - kn + k^2) \\ &\sim 4mn^2 - 4(m+n)n^2/2 + 4n^3/3 \\ &= 2mn^2 - \frac{2}{3}n^3.\end{aligned}$$

If $m \approx n$, it is more efficient than the Gram–Schmidt method, but it is similar to Gram–Schmidt if $m \gg n$.